A Comprehensive Tutorial on SDN-Based Bandwidth Management for a Small Home Wi-Fi Network

Using Mininet-WiFi and Open vSwitch

Project Report

Student: **Sarah Eyad Kamal Hawash**

Instructor: **Dr. Jehad Hamamreh**

Abstract

This report evaluates software-defined networking (SDN) techniques for controlling bandwidth in a small Wi-Fi network emulated with Mininet-WiFi and Open vSwitch (OVS). TCP throughput was measured with iperf on port 5001. In the latest controlled single-client reruns, the baseline (with no TBF qdisc and no OVS QoS attached to the server-facing port) measured 10.4, 10.4, and 10.5 Mbps (mean 10.43 Mbps). TBF alone, configured at 6 Mbps, measured 5.70, 5.72, and 5.72 Mbps (mean 5.71 Mbps). OVS QoS alone, using a 6 Mbps parent and Queue 10 configured with a 4 Mbps minimum and maximum rate, measured 3.82, 3.81, and 3.81 Mbps (mean 3.81 Mbps). One earlier combined TBF-plus-OVS-QoS measurement was 5.72 Mbps; because it was a single run, it is reported separately and not treated as a repeated-sample mean. The experiment also includes a separate two-client queueing test recorded earlier, where each client reported 2.87 Mbps in three simultaneous runs. These results describe the tested emulated setup only; they do not guarantee performance on a physical home Wi-Fi network.

Keywords

Software-defined networking; Mininet-WiFi; Open vSwitch; QoS; HTB; TBF; OpenFlow; bandwidth management; Jain fairness index.

1. Introduction

A small home Wi-Fi network may carry traffic with different bandwidth needs. Without explicit traffic management, concurrent flows compete for available capacity and may receive unequal throughput. Software-defined networking separates control logic from packet forwarding and allows forwarding behavior to be programmed through flow rules. Open vSwitch supports OpenFlow-based forwarding and QoS queue configuration, while Mininet-WiFi provides an emulated wireless environment for repeatable experiments.

The goal of this project is to demonstrate and evaluate practical bandwidth control using two complementary mechanisms: Linux TBF traffic shaping and OVS QoS queues. The multi-client experiment further tests whether individual clients can be assigned to separate queues and receive similar measured throughput during simultaneous TCP transfers.

2. Objectives

- Build and verify a small wireless topology with two clients and one server.
- Measure a baseline TCP throughput without the configured bandwidth limit.
- Apply a 6 Mbps TBF limit and compare the resulting throughput with the baseline.
- Configure OVS QoS with a 6 Mbps parent maximum and per-client queues capped at 3 Mbps.
- Use OpenFlow rules to map each client's TCP traffic to its own queue.
- Repeat simultaneous tests and calculate throughput totals and Jain's fairness index.
- Document limitations and provide a reproducible basis for future experiments.

3. Background

3.1 Software-defined networking and OpenFlow

SDN enables programmable forwarding through flow rules. In this experiment, rules match TCP traffic by source address, destination address, and destination port. The actions set a queue identifier and output the packet through the server-facing OVS port. A lower-priority NORMAL rule remains as a default forwarding rule for traffic that does not match the specific experiment rules.

3.2 Quality of Service and queueing

QoS mechanisms manage traffic treatment and rate constraints. OVS can attach a QoS object to a port and associate queue identifiers with queue objects. The experiment uses the linux-htb QoS type, a parent maximum rate of 6,000,000 bit/s, and two queue objects with maximum rates of 3,000,000 bit/s each. These configured values describe intended limits; throughput measurements are still required to verify behavior in the running environment.

3.3 TBF shaping

Token Bucket Filter (TBF) is a Linux tool that limits the rate at which traffic leaves a network port (often called traffic shaping). The experiment used a 6 Mbps rate with a configured burst and latency. The observed application throughput is expected to be below or near the configured shaping rate because application throughput includes protocol overhead and depends on the emulated network and measurement conditions.

3.4 Throughput fairness

Throughput is the amount of data delivered each second, reported here in megabits per second (Mbps) using iperf, a network speed-testing tool. For two clients with measured throughputs x1 and x2, Jain's fairness index is:

$$J = (x_1 + x_2)^2 / [2(x_1^2 + x_2^2)]$$

The index ranges from 0.5 to 1 for two non-negative client throughputs when at least one is positive; 1 indicates equal throughput. It describes equality between the measured rates, not whether the rates meet every user's service requirements.

4. Experimental Setup

4.1 Topology

The active experiment uses one OVS switch (s1), one access point (AP1), two clients, and one server. The client IP addresses are 10.0.0.1 and 10.0.0.3; the server IP address is 10.0.0.2. The server listens for iperf TCP tests on port 5001. The OVS server-facing interface is s1-eth2, with OpenFlow port number 2. The QoS object used in the experiment was verified as attached to s1-eth2, and the interface was reported up.

Component	Identifier / value	Role
Client 1	10.0.0.1	First TCP traffic source; assigned Queue 10
Client 2	10.0.0.3	Second TCP traffic source; assigned Queue 20
Server	10.0.0.2:5001	iperf TCP receiver/server
OVS switch	s1	OpenFlow forwarding and QoS queue selection
Server-facing OVS port	s1-eth2 (OpenFlow port 2)	Egress port carrying traffic toward the server
Access point	AP1	Wireless access for the clients

4.2 Traffic generation and QoS configuration

The intended iperf test duration was 10 seconds. However, some terminal outputs showed elapsed times of about 11–15 seconds. For this reason, the measurements are reported as recorded, and the test duration should be treated as a limitation rather than a fixed 10-second measurement for every run.

Configuration element	Configured value
Two-client test: parent OVS QoS type	linux-htb
Two-client test: parent QoS maximum	6,000,000 bit/s (6 Mbps)
Two-client test: Queue 10 maximum	3,000,000 bit/s (3 Mbps)
Two-client test: Queue 20 maximum	3,000,000 bit/s (3 Mbps)
Client 1 OpenFlow match	TCP, src 10.0.0.1, dst 10.0.0.2, dst port 5001
Client 1 action	set_queue:10, output port 2
Client 2 OpenFlow match	TCP, src 10.0.0.3, dst 10.0.0.2, dst port 5001
Client 2 action	set_queue:20, output port 2
Latest single-client OVS QoS Queue 10	min-rate 4,000,000 bit/s; max-rate 4,000,000 bit/s (parent max 6,000,000 bit/s)
Latest single-client OVS flow	priority 100, IPv4 source 10.0.0.1, set_queue:10, output port 2

The corresponding flow rules observed in `ovs-ofctl dump-flows s1` were: `priority=100,tcp,nw_src=10.0.0.1,nw_dst=10.0.0.2,tp_dst=5001,actions=set_queue:10,output:2` and `priority=100,tcp,nw_src=10.0.0.3,nw_dst=10.0.0.2,tp_dst=5001,actions=set_queue:20,output:2`. The output showed increasing packet and byte counters for both specific rules.

5. Methodology

1. Verify connectivity between the clients and server using the Mininet-WiFi topology and ping tests.
2. Measure the baseline with the TBF qdisc removed and OVS QoS detached from the server-facing port; repeat the single-client test three times.
3. Apply TBF shaping at 6 Mbps with OVS QoS detached and the Queue 10 OpenFlow rule removed; repeat the single-client test three times.
4. For the separate single-client OVS QoS test, attach linux-htb QoS to s1-eth2 with a 6 Mbps parent maximum and Queue 10 configured with min-rate and max-rate of 4 Mbps; remove TBF and direct matching client traffic to Queue 10.
5. Create Queue 20 capped at 3 Mbps and associate it with the same QoS object.
6. Install OpenFlow rules mapping Client 1 to Queue 10 and Client 2 to Queue 20.
7. Run simultaneous iperf TCP transfers from both clients and record each reported throughput.
8. Inspect TBF statistics, OVS QoS queue counters, and OpenFlow flow counters after the measurements. Counter values are cumulative unless deltas are explicitly collected.

6. Results

6.1 Single-client results

Scenario	Measured speeds (Mbps)	Average (Mbps)	Notes
Baseline, no configured cap	10.4, 10.4, 10.5	10.43	Three measurements; TBF removed and OVS QoS detached
TBF alone, configured at 6 Mbps	5.70, 5.72, 5.72	5.71	Three measurements; OVS QoS detached and Queue 10 flow removed
OVS QoS alone	3.82, 3.81, 3.81	3.81	Three measurements; 6 Mbps parent; Queue 10 min/max 4 Mbps; TBF removed
TBF + OVS QoS	5.72 (one measurement)	—	Single run only; not directly comparable to repeated means

*The latest baseline mean is based on three new measurements. The older multi-client results in Section 6.2 are from a separate earlier configuration and are not mixed into the latest single-client comparison.

6.2 Simultaneous two-client results

Run	**Client 1 (Mbps)**	**Client 2 (Mbps)**	Total speed (Mbps)	Fairness score
1	2.87	2.87	5.74	1.000
2	2.87	2.87	5.74	1.000
3	2.87	2.87	5.74	1.000
Mean of recorded runs	2.87	2.87	5.74	1.000

In the separate earlier simultaneous two-client test, the combined reported throughput of 5.74 Mbps was approximately 95.7% of the configured 6 Mbps parent maximum. Each client reported 2.87 Mbps with a configured 3 Mbps queue maximum. These earlier results are presented separately from the latest single-client reruns because they use a different traffic pattern and queue configuration.

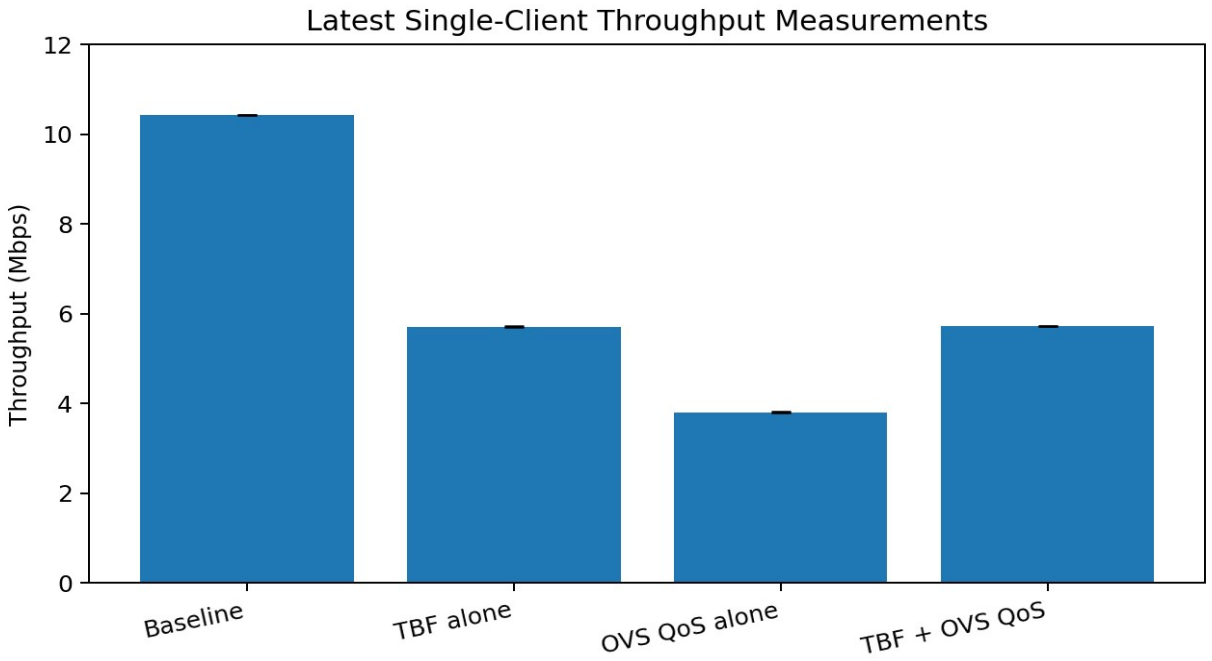


Figure 1. Latest single-client throughput comparison. Means are calculated from three measurements for Baseline, TBF alone, and OVS QoS alone; the combined TBF-plus-OVS-QoS case is a single measurement.

6.3 Fairness calculation

For each simultaneous run, x_1 = 2.87 Mbps and x_2 = 2.87 Mbps. Substituting into Jain's index gives $J = (2.87 + 2.87)^2 / [2(2.87^2 + 2.87^2)] = 1.000$. Because both displayed rates are rounded to two decimal places, the calculated index is based on the displayed values; the unrounded rates may differ slightly.

6.4 OpenFlow and queue counters

Counter source	Client / queue	Packets	Bytes	Error counter
OpenFlow flow counters	Client 1 → Queue 10	20,177	30,500,094	Not shown in flow output
OpenFlow flow counters	Client 2 → Queue 20	19,703	29,785,994	Not shown in flow output
Queue statistics	Queue 10	118,070	178,580,684	3,859
Queue statistics	Queue 20	19,402	29,330,280	301

The OpenFlow counters confirm that both client-specific rules matched traffic. Queue counters are cumulative and do not represent only the final three test runs. The meaning and cause of the reported queue `errors` counters were not established in this experiment; they should not be described as TCP packet loss without additional diagnosis. Flow and queue packet counters also need not match because they may count traffic at different processing stages and over different cumulative histories.

7. Discussion

7.1 Rate control

In the latest separated single-client reruns, TBF alone averaged 5.71 Mbps under a configured 6 Mbps rate. OVS QoS alone averaged 3.81 Mbps with Queue 10 configured at 4 Mbps min-rate and max-rate under a 6 Mbps parent. The baseline averaged 10.43 Mbps. These measurements show that the configured mechanisms changed observed TCP throughput in this emulated setup, but they do not establish that one mechanism is

universally superior. In particular, an OVS queue min-rate is a scheduler configuration, not a guarantee that a single TCP flow will achieve that application throughput.

7.2 Per-client queueing and fairness

The simultaneous multi-client runs reported equal throughput of 2.87 Mbps per client. This is consistent with the intended configuration of separate 3 Mbps maximum-rate queues under a 6 Mbps parent QoS object. The reported combined rate of 5.74 Mbps remained below the configured parent maximum. The result supports the conclusion that the tested configuration delivered equal reported throughput in these three runs. It does not establish behavior under different packet sizes, UDP traffic, additional clients, or changing wireless conditions.

7.3 Interpreting the counters

The flow dump showed 20,177 packets for Client 1's rule and 19,703 packets for Client 2's rule, confirming that both rules matched traffic. The queue statistics were much larger and cumulative. This discrepancy is not sufficient by itself to indicate a fault. Future tests should record changes in counter values before and after each run, and inspect Linux qdisc statistics and iperf sender/receiver summaries if the meaning of queue errors is important.

8. Limitations and Threats to Validity

- The latest single-client comparison has three measurements each for baseline, TBF alone, and OVS QoS alone, but only one measurement for the combined TBF-plus-OVS-QoS state. The separate earlier simultaneous two-client experiment is reported independently and uses a different queue configuration.
- The intended iperf test duration was 10 seconds. However, some terminal outputs showed elapsed times of about 11–15 seconds. For this reason, the measurements are reported as recorded, and the test duration should be treated as a limitation rather than a fixed 10-second measurement for every run.
- The displayed throughput values are rounded by iperf. Means and percentage reductions are therefore approximate.
- TBF and OVS queue counters are cumulative over the running experiment and not isolated per test unless changes in counter values before and after each test are recorded. The OVS queue's configured min-rate should not be interpreted as a guaranteed application throughput.
- The test uses TCP and only two clients. Results may differ for UDP, mixed traffic, more clients, different queue policies, or different wireless conditions.
- The recorded results establish behavior for the tested emulated setup, not guaranteed performance on a physical home Wi-Fi network.

9. Recommendations for Further Work

- Repeat the combined TBF-plus-OVS-QoS configuration at least three to five times under identical test durations.
- Record sender and receiver summaries, exact start/end timestamps, and changes in counter values before and after each test for each run.
- Add a shared-queue multi-client comparison to contrast equal per-client queues with a common queue.
- Test unequal queue limits (for example, 4 Mbps and 2 Mbps) to determine whether the configured allocation is reflected in measured throughput.
- Test UDP traffic at controlled sent traffic rates and report loss/jitter separately from TCP throughput.
- Capture Linux traffic-control statistics (`tc -s qdisc`) and check what the queue error counter actually measures before drawing conclusions from error counters.
- Export all per-run measurements to CSV and plot means with indicators showing how much results vary after the combined configuration has been repeated.

10. Conclusion

This project demonstrated a practical bandwidth-management workflow using Mininet-WiFi, Open vSwitch, OpenFlow, Linux TBF, and OVS QoS queues. In the latest isolated single-client measurements, the baseline mean was 10.43 Mbps, TBF alone averaged 5.71 Mbps with a configured 6 Mbps rate, and OVS QoS alone averaged 3.81 Mbps with a 6 Mbps parent and Queue 10 configured at 4 Mbps min/max. One combined TBF-plus-OVS-QoS measurement was 5.72 Mbps. A separate earlier simultaneous two-client test reported 2.87 Mbps per client and a Jain fairness index of approximately 1.000 across three runs. These findings support conclusions only for the tested emulated configurations. More repetitions for the combined case, keeping each test to the same exact duration, and per-run changes in counter values before and after each test would further strengthen reproducibility.

References

[1] B. Lantz, B. Heller, and N. McKeown, “A Network in a Laptop: Rapid Prototyping for Software-Defined Networks,” Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX), 2010.

[2] Open vSwitch Project, “Open vSwitch Documentation,” project documentation. Available: https://docs.openvswitch.org/

[3] Mininet-WiFi Project, “Mininet-WiFi: Emulator for Software-Defined Wireless Networks,” project repository and documentation. Available: https://github.com/intrig-unicamp/mininet-wifi

[4] A. Demers, S. Keshav, and S. Shenker, “Analysis and Simulation of a Fair Queueing Algorithm,” ACM SIGCOMM Computer Communication Review, vol. 19, no. 4, pp. 1–12, 1989.

[5] R. Jain, D.-M. Chiu, and W. Hawe, “A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems,” DEC Research Report TR-301, 1984.

[6] Linux man-pages project, “tc(8) — show / manipulate traffic control settings,” Linux traffic control manual documentation. Available: https://man7.org/linux/man-pages/man8/tc.8.html

[7] ESnet, “iperf — The TCP, UDP, and SCTP network bandwidth measurement tool,” documentation. Available: https://iperf.fr/

Appendix A. Key Commands Used

```
iperf -s -p 5001

iperf -c 10.0.0.2 -p 5001 -t 10

ovs-ofctl dump-flows s1

ovs-ofctl queue-stats s1

ovs-vsctl --columns=name,qos list Port

ovs-vsctl --columns=other_config,queues list QoS

ovs-vsctl list Queue
```

Appendix B. OpenFlow Rules Observed

```
priority=100,tcp,nw_src=10.0.0.1,nw_dst=10.0.0.2,tp_dst=5001,actions=set_queue:10,outpu
t:2

priority=100,tcp,nw_src=10.0.0.3,nw_dst=10.0.0.2,tp_dst=5001,actions=set_queue:20,outpu
t:2

priority=0,actions=NORMAL
```